Veikka Kumpulainen

Generatiivisen tekoälyn haitat ja vaarat ohjelmistokehityksessä

Tieto- ja ohjelmistotekniikan kandidaatintutkielma

12. toukokuuta 2026

Jyväskylän yliopisto

Informaatioteknologian tiedekunta

Tekijä: Veikka Kumpulainen

Yhteystiedot: veikka.e.kumpulainen@student.jyu.fi

Ohjaaja: Sanna Juutinen, Simo Lehtinen

Työn nimi: Generatiivisen tekoälyn haitat ja vaarat ohjelmistokehityksessä

Title in English: The disadvantages and risks of generative artificial intelligence in software development

Työ: Kandidaatintutkielma

Opintosuunta: Tieto- ja ohjelmistotekniikka

Sivumäärä: 17+0

Tiivistelmä: Tässä kandidaatintutkielmassa tarkastellaan generatiivisen tekoälyn haittoja ja vaaroja ohjelmistokehityksessä kirjallisuuskatsauksen avulla. Keskeisiksi ongelmiksi tunnistetaan tekoälyn tuottaman koodin heikko suorituskyky, tietoturvahaavoittuvuudet sekä puutteet laajojen kontekstien hallinnassa. Tulokset korostavat, että tekoälyä tulisi käyttää tukityökaluna, ei itsenäisenä ratkaisijana, ja sen käyttö edellyttää kriittistä arviointia sekä kehittäjien osaamisen vahvistamista.

Avainsanat: generatiivinen tekoäly, ohjelmistokehitys, suuret kielimallit, tietoturva, suorituskyky, kontekstin hallinta

Abstract: This bachelor's thesis examines the disadvantages and risks of generative artificial intelligence in software development through a literature review. The main issues identified include poor performance of AI-generated code, security vulnerabilities, and limitations in handling large contexts. The findings emphasize that AI should be used as a supportive tool rather than an autonomous solution, and its use requires critical evaluation and strengthened developer expertise.

Keywords: generative artificial intelligence, software development, large language models, software security, performance, context management

Jyväskylässä 12. toukokuuta 2026

Kuviot

Kuvio 1. Chat-GPT:n (v. 5.3) generoima koodi. Syöte: Provide a small C program that adds two numbers together. 5

Sisällys

1 JOHDANTO 1

2 MÄÄRITELMIÄ JA MIKSI TUTKITAAN 2
2.1 Määritelmät 2
2.2 Miksi tutkia 2

3 VAARAT JA HUONOT PUOLET OHJELMISTOKEHITYKSESSÄ 4
3.1 Suorituskyky 4
3.2 Tietoturva ja turvallisuus 6
3.3 Kontekstin hallinta 7

4 TEKOÄLYN HAITTOJEN MINIMOIMINEN 9

5 YHTEENVETO 11

LÄHTEET 12

1 Johdanto

Generatiivista tekoälyä käytetään nykyään paljon ohjelmistokehityksessä, esimerkiksi koodin tuottamisessa ja testaamisessa (Gu ym. 2025). Työkalut, kuten GitHub Copilot ja muut suuret kielimallit voivat tuoda kehittäjälle hyötyjä tuottavuudessa, mutta niiden käyttöönottoon liittyy myös haittoja ja vaaroja, joita ei vielä täysin ymmärretä (Perry ym. 2023). Erityisesti tekoälyn tuottaman koodin turvallisuus, suorituskyky ja luotettavuus ovat nousseet keskeisiksi huolenaiheiksi.

Tässä kandidaatintyössä tarkastellaan kirjallisuuskatsauksen avulla, millaisia haittoja ja vaaroja generatiivisen tekoälyn käytöstä voi seurata ohjelmistokehityksessä. Aihetta tutkitaan, koska on tärkeää, että kaikki ohjelmistokehityksen parissa työskentelevät ovat tietoisia generatiiviseen tekoälyyn liittyvistä riskeistä. Tutkimus toteutetaan perehtymällä aiheesta tehtyihin aiempiin tutkimuksiin ja muodostamalla niiden pohjalta kokonaiskuva generatiivisen tekoälyn käytön haittapuolista ja vaaroista.

Työn alussa esitellään tutkimuksen kannalta keskeiset käsitteet ja perustellaan, miksi aihe on merkityksellinen ja ajankohtainen. Tämän jälkeen tarkastellaan syvällisemmin tekoälyn tuottaman koodin ongelmia suorituskyvyn, tietoturvan ja kontekstinhallinnan näkökulmista sekä pohditaan keinoja, joilla näitä haittoja voidaan mahdollisesti minimoida. Lopuksi esitetään työn keskeiset johtopäätökset yhteenvedon muodossa.

Tutkimuskysymykset:

- Mitä haittoja ja vaaroja tekoälyn käyttö aiheuttaa ohjelmistokehityksessä?
- Miten tekoälyn haitat voidaan minimoida ohjelmistokehityksessä?

2 Määritelmiä ja miksi tutkitaan

Tässä luvussa esitellään tutkielman kannalta keskeiset käsitteet ja rajataan tutkimuksen aihepiiri. Seuraavaksi perustellaan, miksi aihe on ajankohtainen ja miksi sitä on tärkeä tutkia ohjelmistokehityksen nykyisessä toimintaympäristössä. Luvun jälkeen lukijalla on kokonaiskuva tekoälyn laajasta käytöstä ohjelmistokehityksessä sekä siihen liittyvistä haasteista ja ongelmakohdista.

2.1 Määritelmät

Ohjelmistokehityksellä tarkoitetaan joukkoa toimintoja, jotka keskittyvät ohjelmiston luomiseen, suunniteluun, käyttöönottoon ja tukemiseen. Ohjelmistokehityksen tavoitteena on luoda tuote, joka täyttää käyttäjien tarpeet ja liiketoiminnan tehokkaalla, toistettavalla ja turvallisella tavalla. Ohjelmistokehittäjät, ohjelmoijat ja ohjelmistosuunnittelijat kehittävät ohjelmistoja sarjassa, jota kutsutaan ohjelmistokehityksen elinkaareksi. Tekoälypohjaisia työkaluja ja generatiivista tekoälyä käytetään yhä enemmän tukemaan ohjelmistokehitystiimejä koodin tuottamisessa ja testaamisessa (IBM 2024b).

Generatiivinen tekoäly tarkoittaa syväoppimismalleja, jotka pystyvät tuottamaan monimutkaista alkuperäistä sisältöä, kuten pitkiä tekstikokonaisuuksia, korkealaatuisia kuvia, realistisia videoita tai ääntä sekä muuta sisältöä käyttäjän antaman kehotteen perusteella. Yleisellä tasolla generatiiviset mallit koodaavat yksinkertaistetun esityksen koulutusdatastaan ja hyödyntävät tätä esitystä tuottaakseen uutta sisältöä, joka on samankaltaisia mutta ei identtistä alkuperäisen datan kanssa (IBM 2024a). Tämän takia generatiivista tekoälyä voidaan käyttää ohjelmointitehtävien tukena, kuten koodin generoinnissa, muokkauksessa ja testaamisessa. Tässä tutkielmassa keskitytään generatiiviseen tekoälyyn, erityisesti suuriin kielimalleihin.

2.2 Miksi tutkia

Vuoden 2023 kyselyn mukaan 92 prosenttia Yhdysvalloissa työskentelevistä kehittäjistä hyödynsi tekoälyä töissä ja töiden ulkopuolella (Shani ja GitHub Staff 2023), ja alan kyselyt osoit-

tavat, että yli 65 prosenttia ammattikehittäjistä käyttää jonkinlaista tekoälytukea osana päivittäistä työskentelyä (Rahman ja Mehnaz 2024). Tekoälytyökaluja on otettu käyttöön suurten teknologiayritysten kehitysprosessissa, joissa niiden on raportoitu tuottavan 30–50 prosentin tuottavuusparannuksia sekä merkittäviä lyhennyksiä ohjelmistotuotteiden julkaisuaikoihin. Empiirinen tutkimus ei ole pysynyt perässä arvioidessaan nopean käyttöönoton vaikutuksia kriittisiin ohjelmistotekniikan osa-alueisiin, kuten koodin laatuun, turvallisuuteen ja pitkäaikaiseen ylläpidettävyyteen (Rahman ja Mehnaz 2024). Tämän takia aiheitta on tärkeä tutkia, koska tekoälyä käytetään paljon ja sen vaikutuksia kriittisiin ohjelmistotekniikan osa-alueisiin, ei ole keretty tutkia tarpeeksi.

Suorituskyky ja tietoturva ovat olennaisia osia ohjelmiston laatua. Suorituskyvyn heikkeneminen voi vaikuttaa sovelluksen reagointinopeuteen, resurssien kulutukseen ja yleiseen käyttökokemukseen (Li ym. 2024). 55–65 prosenttia kehittäjistä pelkää AI-työkalujen tuottavan tietoturvahaavoittuvuuksia. Yleisimpiä huolia ovat epäselvä riskienhallinta, haavoittuva koodi ja riskialttiit koodirakenteet (Shani ja GitHub Staff 2023). Tämän takia jokaisen ohjelmistokehityksen parissa työskentelevän olisi hyvä tietää ainakin tyypillisimmät haitat ja vaarat, joita tekoäly tekee koodiin.

Ohjelmistokehityksessä joudutaan usein käsittelemään erittäin suuria koodikantoja. Esimerkiksi Google ylläpitää yhtä yhteistä versionhallintavarastoa (monorepoa), joka sisältää yli kaksi miljardia riviä koodia ja yli yhdeksän miljoonaa lähdetiedostoa (*Inside Google's 2 Billion Line Codebase Monorepo - Bomberbot* 2024). Kielimallit eivät pysty käsittelemään näin suuria konteksteja. Lisäksi ihmiset päihittävät tekoälyn subjektiivisissa ja laadullisissa arvioissa. Aiempi kokemus, oivallus ja kokonaisvaltainen näkemys ovat ja tulevat olemaan ihmisille ominaisia vahvuuksia. Nämä antavat ihmisille edun epäselvien ja monitulkintaisten tilanteiden käsittelyssä. Intuitiivisten kykyjensä ansiosta ihmiset ovat yhä parempia hahmottamaan kokonaiskuvan laajasti (Jarrahi 2018). Näiden tekijöiden vuoksi tekoälyn ei voida odottaa korvaavan ihmistä ja ihmisen rooli säilyy korvaamattomana, kun käsitellään laajempia kokonaisuuksia ja konteksteja. Tekoäly on kuitenkin tullut jäädäkseen, jonka takia olisi hyvä oppia käyttämään sitä mahdollisimman tehokkaasti ja turvallisesti apuvälineenä ohjelmistokehityksessä työn laadun parantamiseksi.

3 Vaarat ja huonot puolet ohjelmistokehityksessä

Tässä luvussa tarkastellaan generatiivisen tekoälyn käytöstä ohjelmistokehityksessä aiheutuvia keskeisiä vaaroja ja haittapuolia. Tarkastelu keskittyy erityisesti suorituskykyyn, tietoturvaan ja kontekstinhallintaan liittyviin ongelmiin, joita tekoälyn tuottama koodi voi sisältää. Luvun tavoitteena on tuoda esiin tutkimuskirjallisuudessa havaittuja riskejä ja havainnollistaa, miksi tekoälyn käyttö edellyttää kriittistä arviointia ohjelmistokehityksessä.

3.1 Suorituskyky

Li ym. (2024) mukaan GitHub Copilotin generoima koodi tuottaa toimivaa koodia, mutta usein häviää suorituskyvyssä ihmisen tekemälle. Syitä tekoälyllä generoidun koodin suorituskyvyn heikkenemiseen on tehottomat funktiokutsut, tehottomat silmukkarakenteet, tehottomat algoritmit ja ohjelmointikielen ominaisuuksien tehoton käyttö. Esimerkkejä näistä on liiallinen rekursiivisten funktioiden käyttö, toistuva merkkijonojen liittäminen +- operaattoreilla, useiden sisäkkäisten silmukoiden käyttö ja toistuva olioiden luominen silmukoissa. Tekoälyllä generoitu koodi valitsee usein vähemmän tehokkaita funktiota, vaikka olisi tarjolla optimaalisempia funktiota tehokkuuden parantamiseksi. Algoritmit voivat olla myös tehottomia, jos tekoälyn generoima koodi ohittaa matemaattisen optimoinnin ja käyttää epätarkoituksenmukaista ehtologiikkaa. Ohjelmointikielen ominaisuuksien tehotonta käyttöä on kielen ominaisuuksien alihyödyntäminen, käyttämättömät muuttujat ja tehoton poikkeuskäsittely (Li ym. 2024). Näiden havaintojen perusteella voidaan päätellä, että generatiivisen tekoälyn generoima koodi voi olla suorituskyvyltään heikkoa. Tällöin tekoälyn tuottama koodi on usein riittävää prototyyppien tai pienten tehtävien toteuttamiseen, mutta isommissa järjestelmissä voi syntyä merkittäviä tehokkuusongelmia.

Kielimallit tuottavat usein rakenteellisesti liian monimutkaisia ratkaisuja. Ne välttelevät abstraktointia ja rakentavat koodia ”copy-paste-tyylisesti, eli monistamalla koodia eri paikkoihin hyödyntämättä abstraktointia, mikä lisää toisteisuutta ja hidastaa ohjelman suorittamista (Gu ym. 2025). Tämä johtaa hitaampaan ja tehottomampaan ohjelmakoodiin. Nykyiset suuret kielimallit ovat optimoitu ensisijaisesti ratkaisemaan lyhyitä, funktiotason tehtäviä, eivätkä

ne kykene huomioimaan projektitason optimointia tai laajojen koodirakenteiden yhteensopivuutta (Gu ym. 2025). Suorituskyvyn heikkenemistä lisää, että esimerkiksi Copilotin taustalla olevat algoritmit on optimoitu ensisijaisesti niin, että käyttäjä hyväksyy ehdotuksen, eikä siihen, että koodi olisi turvallista, tehokasta ja helppoa ylläpitää Fu ym. (2025) tutkimuksen mukaan. Malli pyrkii tuottamaan koodia, joka näyttää uskottavalta ja ”oikealta”, vaikka se ei välttämättä olisi tehokasta ja laadukasta. Tämä voi johtaa ratkaisuihin, joissa koodi toimii pintatasolla, mutta sisältää suorituskykyä heikentäviä rakenteita ja ratkaisuja.

Kuviossa 1 on esimerkki Chat-GPT:n 5.3 genoroimasta vastauksesta, kun sille annettiin syöte: ”Provide a small C program that adds two numbers together”.

```c
#include <stdio.h>

int main() {
    int num1, num2, sum;

    // Ask the user for input
    printf("Enter first number: ");
    scanf("%d", &num1);

    printf("Enter second number: ");
    scanf("%d", &num2);

    // Calculate sum
    sum = num1 + num2;

    // Display result
    printf("Sum = %d\n", sum);

    return 0;
}
```

Kuvio 1. Chat-GPT:n (v. 5.3) generoima koodi. Syöte: Provide a small C program that adds two numbers together.

Ohjelma lukee kaksi lukua, joiden yhteenlasku voi tuottaa arvon, joka ylittää ”int-tyypin esityskyvyn. Tämä voi johtaa kokonaislukujen ylivuotoon. Kuviossa 1 oleva esimerkki havainnollistaa hyvin, kuinka kielimallin tuottama koodi voi sisältää kriittisiä virheitä. Lisäksi Tihanyi ym. (2023) tutkimuksessa käsiteltiin esimerkin ohjelmaa ja todettiin, että kielimalli ei osaa ratkaista ongelmaa, vaikka siltä pyydettäisiin turvallinen versio. Kielimalli pystyi korjaamaan ohjelman, vasta kun sille annettiin ESBMC-moduulin tuottama tuloste. ESBMC-moduulin

tuottama tuloste kertoo, täyttyvätkö ohjelmalle määritellyt ominaisuudet, kuten muistiturvallisuus, taulukon rajat ja osoittimien käyttö (*ESBMC,* n.d.)

3.2 Tietoturva ja turvallisuus

Ohjelmistokehittäjille on erittäin yleistä hakea yleistä koodausapua suurilta kielimalleilta, kuten ChatGPT:ltä tai GitHub Copilotilta. Seuraavaksi esiteltävien tutkimusten mukaan kielimallit suoriutuvat kuitenkin heikosti turvallisuuteen liittyvien haavoittuvuuksien tunnistamisessa. Vuonna 2023 tehdyssä tutkimuksessa havaittiin, että tekoälyä käyttävät osallistujat olivat useammin taipuvaisia tuottamaan tietoturvahaavoittuvuuksia sisältävää koodia verrattuna niihin, jotka suorittivat tehtävät ilman tekoälyn apua suurimmassa osassa heille annettuja ohjelmointitehtäviä (Perry ym. 2023). Tekoälyavusteinen koodaus voi heikentää kehittäjän valppautta ja lisätä virheiden syntymistä.

Tekoälyn käytön vaikutuksista tietoturvaan on saatu myös merkittäviä tuloksia. Rahman ja Mehnaz (2024) mukaan tekoälyavusteinen koodaus tuottaa keskimäärin 23,7 prosenttia enemmän tietoturvahaavoittuvuuksia kuin ilman tekoälyn apua tuotettu koodi. Kriittiset haavoittuvuudet lisääntyivät 89 prosenttia ja korkean vakavuuden haavoittuvuudet 47 prosenttia tekoälyä hyödyntäessä. Koodikatselmuksissa kuitenkin 76 prosenttia tekoälyn tuottamista haavoittuvuuksista jäi kehittäjiltä huomaamatta, kun ihmisen tuottaman koodin katselmuksessa vastaava luku oli 52 prosenttia. Tämä osoittaa, että kehittäjät luottavat tekoälyn tuottamaan koodiin enemmän kuin ihmisen kirjoittamaan, vaikka tutkimusnäyttö antaa syyn päinvastaiseen.

GPT-3.5-turbo tuotti 112 000 itsenäisesti käännettävässä C-ohjelmassa 197800 haavoittuvaa funktiota, jotka havaittiin Tihanyi ym. (2023) tutkimuksessa. Lisäksi 29,5 prosenttia GitHub Copilotin tuottamasta Python-koodista ja 24,2 prosenttia JavaScript-koodista sisälsi tietoturvahaavoittuvuuksia Fu ym. (2025) tekemän tutkimuksen mukaan. Tutkimuksessa löytyi myös kahdeksan sellaista haavoittuvuutta, jotka ovat CWE Top 25 (2023) -listalla. CWE-lista on kansainvälinen tietoturvaheikkouksien luokittelujärjestelmä. Lisäksi Sajadi ym. (2025) tutkimuksessa havaittiin, että kolme suurta kielimallia saavuttivat 12,6–40 prosentin tunnistusasteen haavoittuvuuksien tarkassa tunnistamisessa ja käyttäjien varoittamisessa. Juuri maini-

tut empiiriset tutkimukset osoittavat, että generatiivinen tekoäly tuottaa huomattavan määrän haavoittuvaa koodia ja jättää myös huomauttamatta käyttäjän tarjoamassa koodissa olevista tietoturvariskeistä. Tämä korostaa kielimallien rajoitteita tietoturvan analysoinnissa ja ihmisen roolia ohjelmistonkehitysprosessissa. Jos tekoälyyn luottaa sokeasti, niin virheitä tulee erittäin suurella todennäköisyydellä.

Yksi keskeinen syy haavoittuvuuksien syntymiseen on kielimallien koulutusdata. Suuret kielimallit on koulutettu miljardeilla julkisilla avoimen lähdekoodin riveillä, jotka sisältävät turvattomia koodauskäytäntöjä (Fu ym. 2025). GitHub Copilotin taustalla oleva Codex-malli on esikoulutettu GitHubista peräisen olevalla epäluotettavalla datalla, jonka takia se voi tuottaa turvattomia koodinpätkiä ja GPT-malli on koulutettu avoimella koodilla, jossa esiintyy väärin toteutettuja ratkaisuja, haavoittuvia funktioita ja virheellisiä suunnittelumalleja (Fu ym. 2025; Tihanyi ym. 2023). Kun haavoittuvaa koodia käytetään projekteissa, se päätyy takaisin mallien treenidataan ja syntyy kierre, jossa generatiiviset mallit tuottavat yhä lisää haavoittuvaa koodia (Fu ym. 2025).

Kielimallit tuottavat myös epätäydellistä, haavoittuvaa tai väärää koodia, vaikka testit menisivät läpi. Ne eivät osaa priorisoida turvallisuutta, koska koulutusdata sisältää paljon huonolaatuista ja haavoittuvaa koodia (Gu ym. 2025). Lisäksi tekoäly toistaa koodia mieluummin kuin rakentaa hyvälaatuisia rakenteita. (Gu ym. 2025) Tämän takia tekninen velka kasvaa nopeasti.

3.3 Kontekstin hallinta

Kielimalleilla on haasteita ylläpitää laajoja ja monimutkaisia konteksteja ohjelmistokehityksessä. Mallit toimivat rajatulla syötteellä, eivätkä ne pysty hahmottamaan kokonaista järjestelmäkokonaisuutta yhdellä kertaa. Tämä rajoittaa mallien kykyä tuottaa johdonmukaisia kokonaisarkkitehtuureja tai ymmärtää pitkän kehitysprosessin riippuvuuksia. Kielimallit vaativat usein tehtävien jakamista pieniin osiin, mikä lisää riskiä, että ratkaisuista tulee epäyhtenäisiä, ristiriitaisia tai väärällä abstraktiotasolla toimivia (Belzner, Gabor ja Wirsing 2023). Kielimallit ei useinkaan ymmärrä projektin kokonaista arkkitehtuuria, riippuvuuksia ja sitä, miten yksittäinen funktio vaikuttaa muualla (Fu ym. 2025). Tekoäly ymmärtää koodia vain paikal-

lisesti, ei projektitasolla. Se ei hahmota ohjelmiston arkkitehtuuria tai kokonaislogiikkaa ja epäonnistuu usein pitkissä riippuvuuksissa ja monimutkaisissa koodipoluissa. (Gu ym. 2025)

Kontekstin hallinnan ongelmia korostaa myös kielimallien päätöksenteon läpinäkymättömyys. Lähes puolet kehittäjistä kokee, etteivät he ymmärrä, miksi tekoäly valitsee tietyn ratkaisun, mikä vaikeuttaa sen tuottaman koodin arviointia ja luotettavuutta (Shani ja GitHub Staff 2023). Myös käyttäjien antamat promptit ovat usein epätäydellisiä ja kielimallit täyttävät puuttuvat kohdat omilla oletuksillaan, jotka ovat usein vääriä. Monimutkaisissa projekteissa pienikin väärä oletus voi paisua ketjureaktioksi (Gu ym. 2025).

Kielimallit eivät myöskään kykene havaitsemaan omia tuottamiaan haavoittuvuuksia tai arvioimaan ratkaisujensa turvallisuutta kokonaisvaltaisesti. Se kuitenkin korjaa virheitä, jos saa formaalin analyysityökalun, kuten ESBMC:n, tuottaman counterexample-raportin. Tällainen raportti kertoo missä virhe tapahtuu ja miten se voidaan toistaa (Tihanyi ym. 2023). Tämä osoittaa, että ilman oikeaa työkalua kielimallit eivät ymmärrä virheiden lähteitä tai laajempaa kontekstia, joka olisi tärkeää monimutkaisten ohjelmistojen turvallisen ja johdonmukaisen kehittämisen kannalta.

4 Tekoälyn haittojen minimoiminen

Ihmisen asiantuntemuksella on korvaamaton rooli tekoälyn tukemassa ohjelmistokehityksessä. Tekoälyyn tulisi suhtautua yhteisötyökumppanina eikä autonomisena koodin tuottajana. Kehittäjien tulee säilyttää vastuu turvallisuuden varmistamisesta (Shukla, Joshi ja Syed 2025). Ihminen ja tekoäly yhdessä tuottavat parempaa laatua, kuin kumpikaan yksin (Jarrahi 2018). Tekoälyyn ei saa luottaa ilman kriittistä tarkastelua, sillä sen tuottamat ratkaisut voivat sisältää puutteita, joita tekoäly ei itse pysty havaitsemaan.

Merkittävät osa kehittäjistä kokee, ettei heillä ole riittävästi osaamista tekoälyn turvalliseen käyttöön. Shani ja GitHub Staff (2023) mukaan 50–70 prosenttia kehittäjistä raportoi koulutusvajetta tekoälyn turvalliseen käyttöön, mikä lisää virheiden ja tietoturvariskien todennäköisyyttä. Ohjelmistokehityksen parissa työskentelevien organisaatioiden on otettava käyttöön prosesseja tietoturvatarkastuksille, syventävälle koulutukselle ja automatisoidulle tietoturva-analyysille, jotta ne tunnistaisivat tekoälyn tuottamaan koodiin liittyviä riskejä (Rahman ja Mehnaz 2024). Taitotason merkitys korostuu myös kehittäjäkokemuksen mukaan. 89 prosenttia juniorikehittäjistä hyväksyivät tekoälyn ehdotukset sellaisenaan, kun taas seniorikehittäjillä vastaava luku oli 62 prosenttia ja he tuottivat turvallisempaa koodia (Rahman ja Mehnaz 2024). Tämä kertoo, että kokematon kehittäjä on alttiimpi virheille ja liialliselle luottamukselle tekoälyyn.

Tekoälyn tuottaman koodin laatua voidaan parantaa vaikuttamalla mallille annettuihin kehotteisiin. Esimerkiksi huolellisesti suunnitellut promptit voivat parantaa tekoälyn tuottaman koodin suorituskykyä (Li ym. 2024). Perry ym. (2023) tutkimuksessa on havaittu, että osallistujat, jotka panostivat enemmän tekoälylle antamiensa promptien muotoiluun, tuottivat todennäköisemmin turvallisia ratkaisuja. Myös Sajadi ym. (2025) tutkimuksessa huomattiin, että kun promptille antoi ohjeen ”address security vulnerabilities”(huomioi tietoturvahaavoittuvuudet), parani kielimallien kyky havaita haavoittuvuuksia merkittävästi. Kun Copilotille annetaan staattisen analyysityökalun varoitusviesti, korjaukset paranevat. Pelkästään antamalla kielimallille syötteeksi ”fix”(korjaa) korjaantui 19,3 prosenttia tietoturvaongelmista, ja parannetulla syötteellä korjausaste nousi 55,5 prosenttiin (Fu ym. 2025). Pelkkä yksinkertainen korjauskomento ei riitä, vaan jotta mallit korjaavat koodia paremmin, pitää niitä ohjata

täsmällisillä varoitusviesteillä.

5 Yhteenveto

Tässä kandidaatintutkielmassa tarkasteltiin generatiivisen tekoälyn haittoja ja vaaroja ohjelmistonkehityksessä. Tavoitteena oli selvittää, mitä haittoja ja vaaroja syntyy, kun tekoälyä käytetään, ja miten voidaan minimoida haitat ohjelmistonkehityksessä. Tutkimus toteutettiin kirjallisuuskatsauksena hyödyntäen ajankohtaisia tieteellisiä tutkimuksia ja raportteja.

Kirjallisuuskatsauksen perusteella keskeisimmät tekoälyn haitat ja vaarat ohjelmistonkehityksessä liittyvät tekoälyn tuottaman koodin suorituskykyyn, tietoturvaan ja kykyyn hallita laajoja kontekseja. Tekoälyn tuottama koodi oli useiden tutkimusten mukaan tehotonta ja sisälsi ylimääräisiä rakenteita ja epätarkoituksenmukaisia ratkaisuja. Lisäksi tekoälyn tuottamasta koodista löytyi huomattavasti enemmän haavoittuvuuksia, ja ne olivat vaikeammin havaittavissa kuin ihmisen kirjoittamasta koodista. Kielimallit toimivat myös rajallisesti, kun kontekstin koko kasvoi. Tekoälyn ratkaisut olivat huonolaatuisia kokonaisarkkitehtuurin kannalta ja piilevää teknistä velkaa syntyi helposti.

Haittojen minimoimiseksi todettiin, että olisi hyvä ymmärtää tekoäly kehittämisen työkaluna eikä autonomisena koodin tuottajana. Tutkimukset osoittivat myös, että moni ohjelmistokehittäjä koki tekoälyn käytön koulutuksen puutteelliseksi. Oikeanlaisilla prompteilla pystyi vaikuttamaan huomattavasti tekoälyn tuottaman koodin laatuun.

Jatkossa olisi tärkeää tutkia tekoälyn käytön vaikutuksia pitkällä aikavälillä koodin ylläpidettävyyteen, tietoturvaan ja teknisen velan kasvuun. Tulokset korostavat tarvetta kehittää käytäntöjä ja työkaluja, joilla tekoälystä saadaan paras hyöty irti minimoiden samalla haitat ja vaarat.

Lähteet

Belzner, Lenz, Thomas Gabor ja Martin Wirsing. 2023. “Large Language Model Assisted Software Engineering: Prospects, Challenges, and a Case Study Preprint” [kielellä en].

ESBMC [kielellä en]. n.d. Viitattu 4. toukokuuta 2026. http://esbmc.github.io/.

Fu, Yujia, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu ja Jinfu Chen. 2025. *Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study.* ArXiv:2310.02059 [cs], helmikuu. Viitattu 2. maaliskuuta 2026. https://doi.org/10.48550/arXiv.2310.02059. http://arxiv.org/abs/2310.02059.

Gu, Alex, Naman Jain, Wen-Ding Li, Manish Shetty, Yijia Shao, Ziyang Li, Diyi Yang, Kevin Ellis, Koushik Sen ja Armando Solar-Lezama. 2025. *Challenges and Paths Towards AI for Software Engineering.* ArXiv:2503.22625 [cs], maaliskuu. Viitattu 11. helmikuuta 2026. https://doi.org/10.48550/arXiv.2503.22625. http://arxiv.org/abs/2503.22625.

IBM. 2024a. *What Is Artificial Intelligence (AI)? | IBM* [kielellä en], elokuu. Viitattu 11. helmikuuta 2026. https://www.ibm.com/think/topics/artificial-intelligence.

———. 2024b. *What Is Software Development? | IBM* [kielellä en], marraskuu. Viitattu 11. helmikuuta 2026. https://www.ibm.com/think/topics/software-development.

Inside Google's 2 Billion Line Codebase Monorepo - Bomberbot [kielellä en-US]. 2024. Section: Tech, huhtikuu. Viitattu 26. maaliskuuta 2026. https://www.bomberbot.com/tech/inside-googles-2-billion-line-codebase-monorepo/.

Jarrahi, Mohammad Hossein. 2018. “Artificial Intelligence and the Future of Work: Human-AI Symbiosis in Organizational Decision Making”. *Business Horizons* 61 (heinäkuu). https://doi.org/10.1016/j.bushor.2018.03.007.

Li, Shuang, Yuntao Cheng, Jinfu Chen, Jifeng Xuan, Sen He ja Weiyi Shang. 2024. “Assessing the Performance of AI-Generated Code: A Case Study on GitHub Copilot”. Teoksessa *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE),* 216–227. ISSN: 2332-6549. Lokakuu. Viitattu 7. helmikuuta 2026. https://doi.org/10.1109/ISSRE62328.2024.00030. https://ieeexplore.ieee.org/document/10771230/.

Perry, Neil, Megha Srivastava, Deepak Kumar ja Dan Boneh. 2023. “Do Users Write More Insecure Code with AI Assistants?” Teoksessa *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security,* 2785–2799. CCS ’23. New York, NY, USA: Association for Computing Machinery, marraskuu. ISBN: 979-8-4007-0050-7, viitattu 2. maaliskuuta 2026. https://doi.org/10.1145/3576915.3623157. https://dl.acm.org/doi/10.1145/3576915.3623157.

Rahman, Parvejur ja Sagufta Mehnaz. 2024. “International Journal for Multidisciplinary Research (IJFMR)” [kielellä en]. *SSRN Electronic Journal,* ISSN: 1556-5068, viitattu 24. helmikuuta 2026. https://doi.org/10.2139/ssrn.5054029. https://www.ssrn.com/abstract=5054029.

Sajadi, Amirali, Binh Le, Anh Nguyen, Kostadin Damevski ja Preetha Chatterjee. 2025. “Do LLMs consider security? an empirical study on responses to programming questions” [kielellä en]. *Empirical Software Engineering* 30, numero 4 (heinäkuu): 101. ISSN: 1382-3256, 1573-7616, viitattu 9. helmikuuta 2026. https://doi.org/10.1007/s10664-025-10658-6. https://link.springer.com/10.1007/s10664-025-10658-6.

Shani, Inbal ja GitHub Staff. 2023. *Survey reveals AI’s impact on the developer experience* [kielellä en-US], kesäkuu. Viitattu 2. maaliskuuta 2026. https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/.

Shukla, Shivani, Himanshu Joshi ja Romilla Syed. 2025. *Security Degradation in Iterative AI Code Generation – A Systematic Analysis of the Paradox.* ArXiv:2506.11022 [cs] version: 2, syyskuu. Viitattu 24. helmikuuta 2026. https://doi.org/10.48550/arXiv.2506.11022. http://arxiv.org/abs/2506.11022.

Tihanyi, Norbert, Tamas Bisztray, Ridhi Jain, Mohamed Amine Ferrag, Lucas C. Cordeiro ja Vasileios Mavroeidis. 2023. “The FormAI Dataset: Generative AI in Software Security Through the Lens of Formal Verification”. Teoksessa *Proceedings of the 19th International Conference on Predictive Models and Data Analytics in Software Engineering,* 33–43. ArXiv:2307.02192 [cs]. Joulukuu. Viitattu 11. helmikuuta 2026. https://doi.org/10.1145/3617555.3617874. http://arxiv.org/abs/2307.02192.